Lab 14: Implement Solution of Modular Linear Equation

Theory:

A modular linear equation is of the form ax ≡ b (mod n). It can be solved using the Extended Euclidean Algorithm. A solution exists if and only if gcd(a, n) divides b.

Complexity Overview

Resource	Complexity	Condition
Time	O(log(min(a, n)))	Governed by EA iteration
Space	O(log(min(a, n)))	Recursive stack structure

Algorithm

1. Compute d = gcd(a, n) using the Extended Euclidean Algorithm, which also returns x and y.
2. If d does not divide b, no solution exists.
3. If d divides b, calculate the base solution x0 = (x * (b/d)) % n.
4. Generate the d solutions by adding multiples of n/d to x0.

Pseudocode

```
BEGIN
  (d, x, y) ← EXTENDED-EUCLIDEAN(a, n)
  IF b % d == 0 THEN
    x0 ← (x * (b/d)) mod n
    FOR i ← 0 to d-1 DO
      print (x0 + i * (n/d)) mod n
    END FOR
  ELSE
    print 'No solution'
  END IF
END
```

Conclusion:

The experiment successfully applies the Extended Euclidean Algorithm backwards to calculate the Modular Linear Equation constraints structurally. By dynamically identifying the Greatest Common Divisor (GCD) between parameters, the program correctly distinguished between solvable and impossible linear moduli sets.

For the first test case `14x ≡ 30 (mod 100)`, the algorithm accurately computed GCD(14, 100) = 2, which cleanly divides 30. Thus it correctly generated dual discrete solutions dynamically aligned against modulus base limits. Conversely, evaluating `2x ≡ 3 (mod 4)` successfully tripped the divisibility check (GCD of 2 does not divide 3), correctly raising the explicit 'No solution exists' flag and validating algorithmic logic.